V. System Design and Implementation

System Design and Implementation describes the implemented Android system that connects the final image-level classifier and the disease-region instance-segmentation model. The chapter focuses on the executable control flow, model–application contracts, release packaging, runtime configuration, persistence, and maintenance boundaries. Mathematical training definitions remain in Chapter IV, while numerical performance and cross-device runtime outcomes remain in Chapter VI. The application is a preliminary screening prototype and is not a veterinary or laboratory diagnostic system.

1. AI Model Integration

1.1 Overall System Architecture

The system was organized as a layered mobile pipeline rather than as a set of independent screens. The presentation layer is a native Android interface using CameraX/gallery input, Kotlin, Jetpack Compose, Hilt, and view-model orchestration. Image acquisition, preprocessing, model execution, decoding, presentation, and persistence are separated so that each boundary can be validated independently.

At the beginning of a screening session, the user captures or selects an image. The bitmap is validated, resized and normalized for the classifier, then executed through TensorFlow Lite/LiteRT. The classifier returns four scores in the fixed order Healthy, BG, WSSV, and WSSV_BG, together with confidence and ranked alternatives.

The deployed application uses a conditional two-model cascade. The FP32 classifier `yolo26m_asl_ldam_simam_dcfr_fp32.tflite` runs for every accepted image. If the output is Healthy above the application threshold, or if the confidence is below threshold, the workflow terminates after classification and no disease mask is generated. If BG, WSSV, or WSSV_BG is produced above threshold, the application invokes the FP16 instance-segmentation artifact `yolo11n_seg_float16.tflite`. This dispatch rule is an application-control decision; it must not be interpreted as verified disease ground truth.

The final segmentation model integrated into this branch is YOLO11n-seg with Coordinate Attention followed by SimAM at P3/P4/P5. Its mobile output semantics remain two mask classes, BG and WSSV. Healthy is not a segmentation class, and WSSV_BG remains an image-level classification category rather than a third mask label.

The result object stores the checked image, predicted class, confidence, ranked alternatives, mask data when generated, and runtime metadata. Results are saved only after an explicit user action and pass through the prediction-log repository, where duplicate saves in the same session are prevented. Cloud synchronization is separated from local inference, so Firebase availability does not determine the locally produced model output.

Figure 5.1 summarizes the implementation boundaries and data dependencies. The diagram distinguishes on-device inference from cloud-backed synchronization and shows that segmentation is conditionally dispatched from the disease-decision stage rather than implemented as a separate research screen. The architecture also defines failure isolation. Camera/gallery failures are handled before tensor cre-

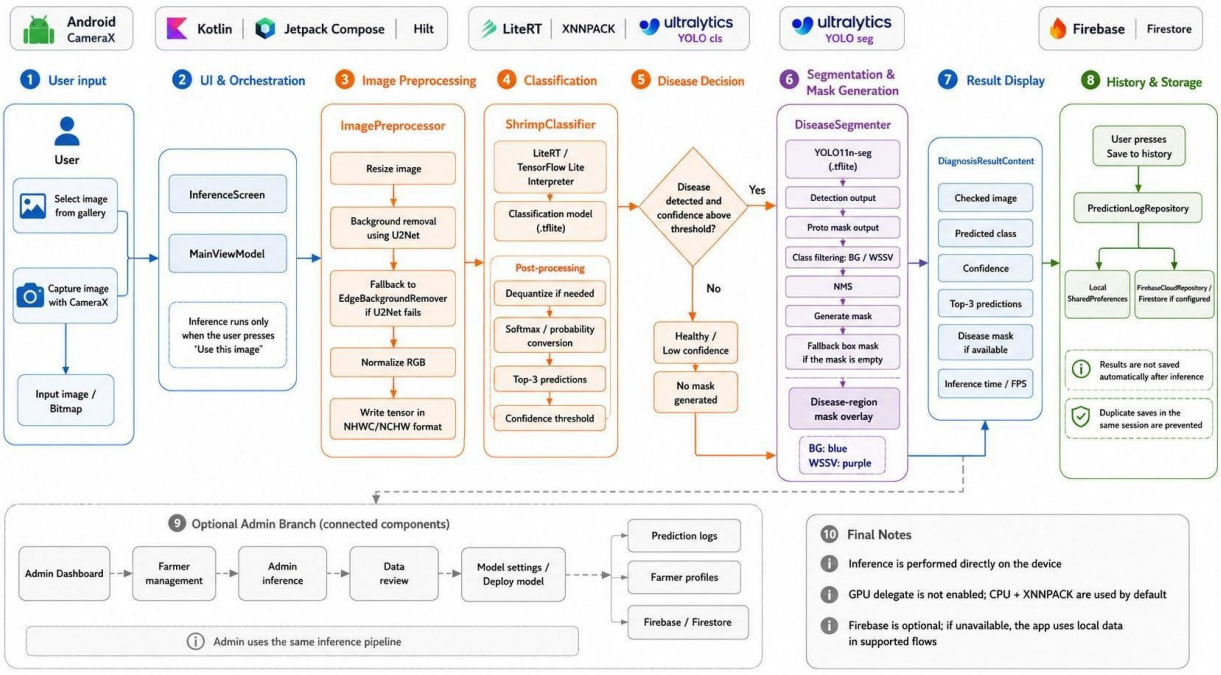


Figure 5.1. Mobile AI pipeline from camera or gallery input through Android orchestration, image preprocessing, on-device classification, disease-conditioned segmentation, result presentation, history storage, and administrator-side review.

ation; preprocessing failures do not invoke the classifier; classification failures do not dispatch segmentation; segmentation-decoder failures preserve a valid classification result; and Firebase interruptions do not change the model output produced locally.

1.2 Classification Model Integration

The mobile classification path uses YOLO26m-cls trained with ASL-LDAM and SimAM-DCFR. ASL-LDAM is training-only and is not part of the mobile inference graph. The configured deployment artifact is `yolo26m_asl_ldam_simam_dcfr_fp32.tflite`, with input tensor [1,224,224,3], output tensor [1,4], and the fixed class order Healthy, BG, WSSV, WSSV_BG.

The final classifier's application contract is verified at the artifact/tensor level rather than by reusing the parameter count or checkpoint size of the plain YOLO26m reference. Because SimAM-DCFR contains trainable texture and channel-gating branches, the reference 10.36M-parameter value reported for the unmodified candidate is not silently reassigned to the final exported classifier. Exact final-artifact complexity metadata are reported only when directly audited from the released model.

The Android preprocessing path must preserve spatial resolution, RGB channel order, tensor layout, and value-range/normalization parity with the exported network. The confidence threshold is an application decision threshold controlling warning text and segmentation dispatch; it is not an offline benchmark threshold.

The preprocessing architecture includes foreground preparation using U2Net with an edge-based fallback. Mobile preprocessing is therefore treated as a deployment contract rather than as a visually approximate operation: a foreground image that looks plausible can still be numerically incompatible if channel order, value range, or background convention differs from training.

CPU execution is used in the common Android configuration. The application exposes four processing threads, XNNPACK/LiteRT CPU acceleration, and a disabled GPU path. These settings improve portability but do not imply identical speed across devices because processor capability, thread scheduling, interpreter initialization, image conversion, preprocessing, postprocessing, and thermal state can differ.

1.3 Instance-Segmentation Integration

The segmentation component provides spatial model evidence for BG and WSSV. Healthy has no positive mask class, and WSSV_BG remains an image-level co-infection category. For a disease-triggered classification, the mask output may contain BG regions, WSSV regions, or both.

The final deployed segmentation artifact is `yolo11n_seg_float16.tflite`, corresponding to YOLO11n-seg + CA→SimAM at P3/P4/P5. The retained model summary in Chapter VI reports 2,854,718 parameters (2.855M), approximately 9.7 GFLOPs, an FP16 TFLite size of 5.65 MB, full-test mask mAP50 of 49.71%, labeled-test mask mAP50 of 55.07%, labeled-test mAP50-95 of 19.64%, and full-test recall of 49.70%. The 9.3 ms/image notebook measurement belongs to a separate GPU timing context and is not an Android latency claim.

The decoder receives detection fields and prototype-mask data, filters BG/WSSV instances, applies non-maximum suppression, reconstructs masks, and overlays them on the source image. Presentation colors are UI metadata only; the numeric class mapping and decoded tensors define the model contract.

End-to-end operation of this final segmentation artifact is supported by the three-device Android runtime study in Chapter VI. Therefore the previous wording that treated final-device segmentation verification as still incomplete is removed. The remaining boundary is narrower: the mobile logs establish operational execution and runtime feasibility, not veterinary correctness or a controlled hardware benchmark.

1.4 Model–Application Interface Contract

A model is safely replaceable only when tensor, precision, semantic, and decoder contracts remain explicit. Training-only operations such as RandAugment and ASL-LDAM are excluded from the runtime contract.

Table 5.1. Model–application interface contract

Interface element	Classification path	Segmentation path	Application responsibility
Image source	CameraX bitmap or gallery bitmap	Validated source image from the same screening session	Decode the image, retain orientation, and reject inaccessible input.
Colour representation	RGB	RGB	Preserve channel order and prevent unintended conversion.
Spatial input	$[1, 224, 224, 3]$	640×640 RGB for the retained final model	Apply model-specific resizing and normalization; retain the source image for overlay.
Precision / artifact	Final classifier TFLite; FP32	Final segmentation TFLite; FP16	Load the approved release bundle rather than independent, unversioned model files.
Output	Four class scores	Detection fields and prototype-mask representation	Validate tensor dimensions and finite values, then decode with the matching implementation.
Class mapping	Healthy, BG, WSSV, WSSV_BG	BG, WSSV	Keep serialized indices synchronized with UI labels, history, and export.

Table 5.1. Model–application interface contract (continued)

Interface element	Classification path	Segmentation path	Application responsibility
Decision product	Predicted label, confidence, and top-three alternatives	Mask instances or label-map overlay for disease-triggered cases	Apply the application threshold and suppress disease-mask output for Healthy or below-threshold results.
Runtime metadata	Model identifier, processing time, and thread setting	Segmentation checkpoint identifier and decoder status when available	Attach traceability fields to the result and to the saved history record.
Failure product	Structured inference error; no fabricated class	Structured mask error while retaining the classification result	Display a bounded message and keep persistence failures separate from inference failures.

The contract treats the classifier and conditional segmenter as one versioned deployment bundle. Classification is executed for every accepted image; segmentation is invoked only for an above-threshold BG, WSSV, or WSSV_BG decision. Healthy and below-threshold results therefore terminate without a disease-mask product. Runtime fields remain application-recorded operational measurements, and the predicted class is used to control dispatch rather than as verified veterinary ground truth.

The Android layer remains responsible for input validation, interpreter execution, semantic decoding, error handling, presentation, persistence, and version traceability. A model file that loads successfully can still be semantically incompatible if class order, normalization, precision assumptions, or mask-decoder interpretation changes.

2. Data Flow and Processing

2.1 Image Acquisition and Validation

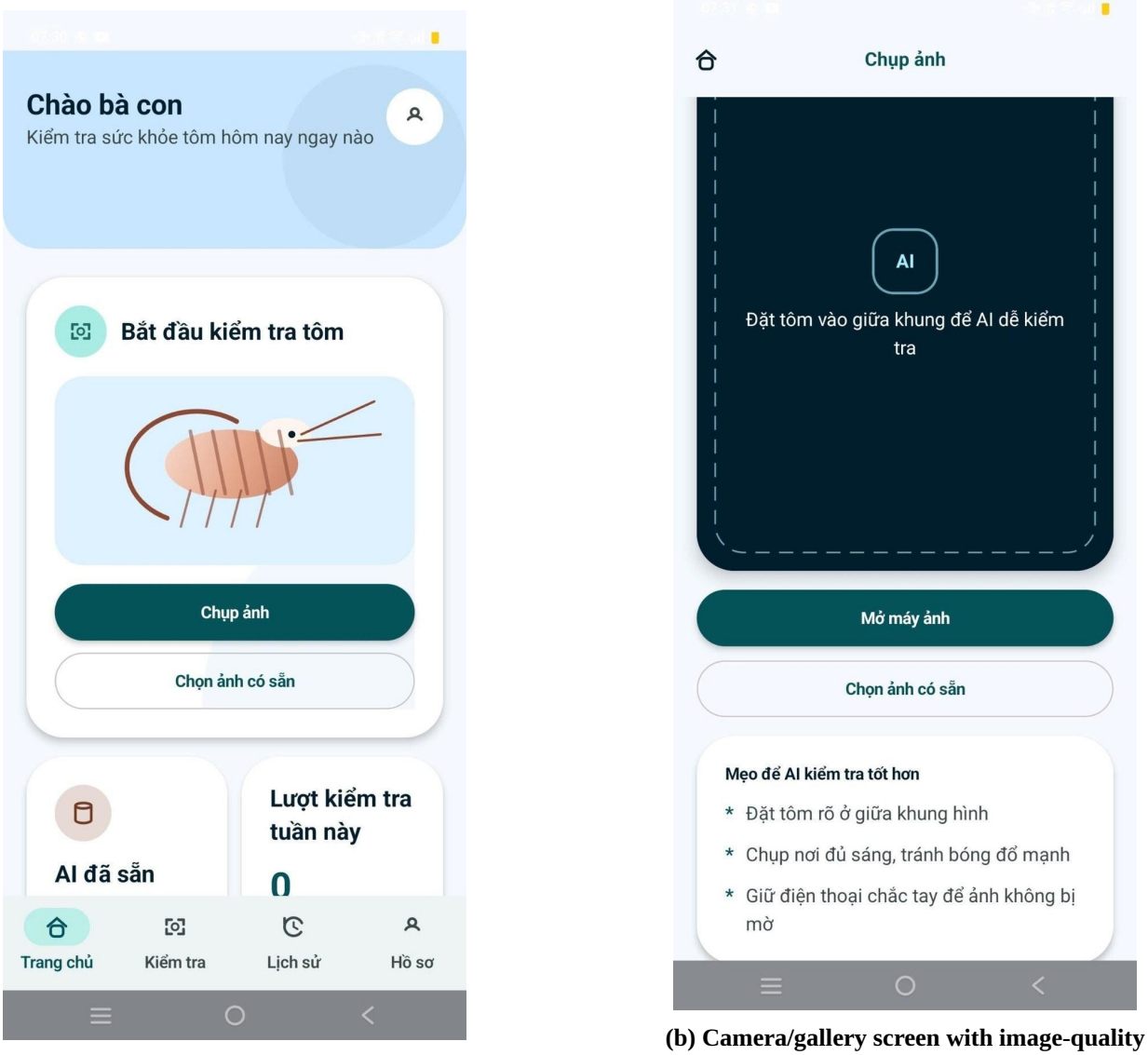
The screening workflow begins from two explicit user actions: capture a new image or select an existing image. The home screen provides both entry points, while the capture/upload screen isolates image selection from inference. This prevents an unintended camera frame or gallery preview from being treated as an accepted model input. Inference begins only after the user confirms the selected image.

The acquisition layer receives a camera result or gallery URI and resolves it into an application-accessible bitmap. Validation occurs before model preprocessing. The application must confirm that the image can be decoded, that dimensions are non-zero, and that the bitmap can be converted into the expected RGB representation. Orientation metadata must be applied consistently when exposed by the Android decoder; otherwise, the displayed orientation can differ from the orientation evaluated by the model.

The capture interface also provides guidance on framing, lighting, blur, and device stability. These controls reduce preventable acquisition failures but do not replace robustness evaluation. Severe blur, clipping, or occlusion can remove disease evidence that downstream preprocessing cannot recover.

image and communicates the acquisition conditions expected by the model.

image-quality guidance.



(a) Home screen and start-check entry point.

(b) Camera/gallery screen with image-quality guidance.

Figure 5.2. Main entry screens of the mobile shrimp disease screening application.

The screen separation also supports error recovery. A failed decode returns the user to image selection without creating a history record. A successfully decoded image becomes the immutable source for both classification and segmentation processing in that screening session. Retaining the source image is necessary because mask decoding and overlay composition must map model-space coordinates back to the displayed image.

2.2 Preprocessing and Inference Orchestration

→ →

test image preprocessing independently from score decoding and to distinguish model latency from image preparation time.

For the configured classifier, preprocessing resized the accepted RGB image to 224 $\times$ 224. The system architecture further records U2Net background removal, an edge-based fallback, RGB normalization, and tensor construction. The fixed input metadata shown by the application is [1,224,224,3], so the active classifier path writes an NHWC tensor. All channel and scaling operations must be deterministic at inference. Training-only augmentation, corruption transformations, ASL-LDAM margin construction, and label smoothing are deliberately absent from this path.

The interpreter stage initializes the model, allocates tensors, writes the prepared input, executes the network, and reads the four output values. Postprocessing converts the output into the probability representation expected by the UI, validates that the values are finite, and constructs a ranked list. If the exported model already returns normalized probabilities, a second softmax would be incorrect; if it returns logits, probability conversion is required. The deployment sanity test therefore checks both the tensor metadata and the behavior on a known image rather than relying on a filename alone.

The segmentation path was designed as validated image $\rightarrow$ segmentation preprocessing $\rightarrow$ selected instance segmentation checkpoint $\rightarrow$ detection/prototype decoding $\rightarrow$ class filtering and NMS $\rightarrow$ mask reconstruction $\rightarrow$ overlay generation. Classification and segmentation may use different input resolutions and normalization rules. The Android orchestration layer must not reuse the classification tensor as the segmentation tensor unless the segmentation export contract explicitly matches it.

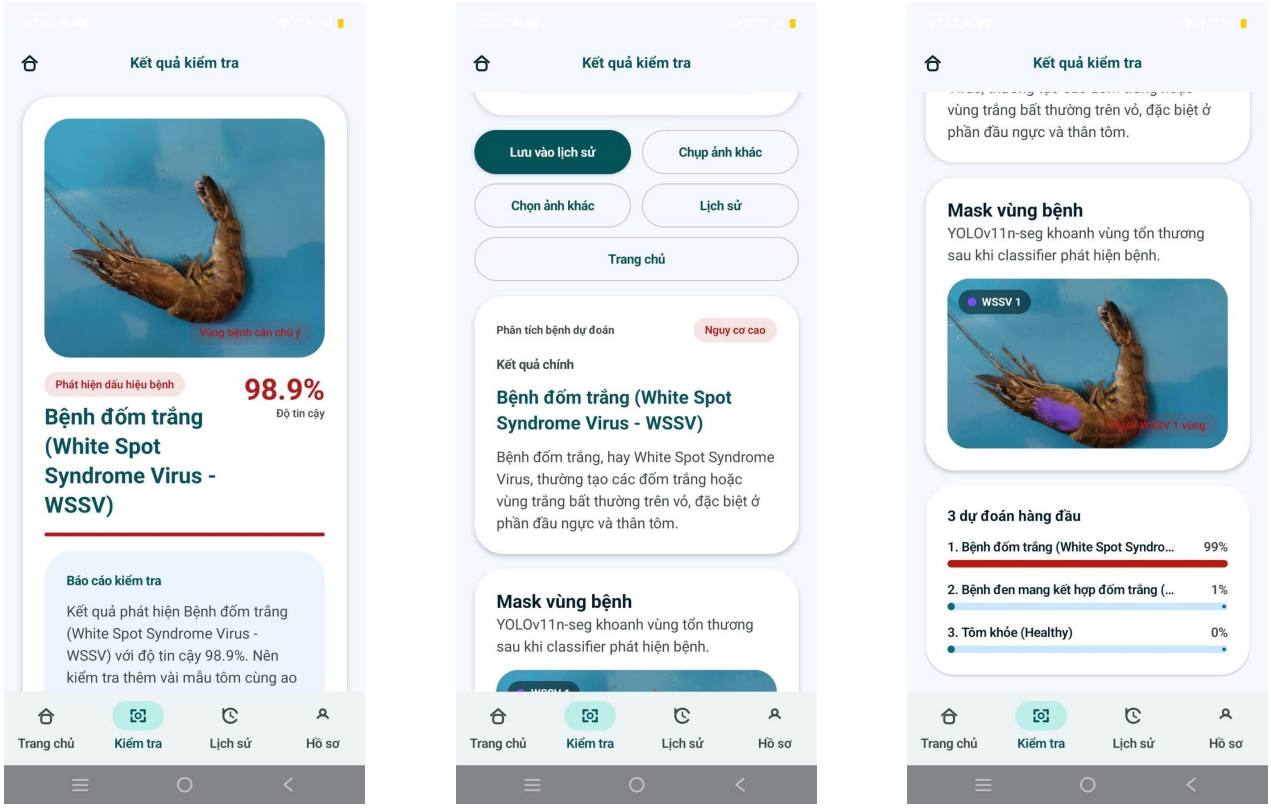
The disease decision lies between the two interpreters. This placement avoids unnecessary segmentation execution for Healthy or low-confidence outputs and enforces the no-fabricated-mask rule. It also means that a segmentation failure does not erase a valid classification result. The result screen can present the image-level prediction with a clear message that spatial evidence was unavailable, rather than substituting an arbitrary region.

2.3 Result Interpretation and Presentation

The result interface converts model outputs into a bounded screening report. The primary card presents the predicted disease label and confidence. A supporting section presents the ranked alternatives, enabling the user to distinguish a concentrated output from a close competition among disease categories. A disease explanation describes the visible condition in non-prescriptive language. The interface also provides actions for saving the record, checking another image, returning to the home screen, or reviewing history.

For diseased cases, the result interface includes the segmentation overlay when available. The mask is presented on the source image rather than as an isolated tensor because spatial evidence is meaningful only in relation to shrimp anatomy. The associated text identifies it as a model-estimated region requiring attention. It must not be described as histopathological confirmation, causal explanation, or a guaranteed boundary. A model can focus on a visually correlated region and still be wrong about the disease class or the mask extent.

Similarly, the ranked alternatives illustrate output decoding but do not replace dataset-level classwise evaluation.



(a) Predicted label and confidence. **(b) Explanation and result actions.** **(c) Disease-region overlay and ranked alternatives.**

Figure 5.3. Result interpretation screens for classification output and disease-region evidence.

Runtime fields can be shown when technical information is enabled. Processing time and FPS are diagnostic observations tied to a specific device, thread configuration, model artifact, and application state. They should not be interpreted as universal mobile benchmarks. Hiding these fields for general users while retaining them for technical inspection reduces interface complexity without removing deployment traceability.

2.4 History, Export, Deletion, and Feedback Flow

History is a user-controlled persistence stage rather than an automatic side effect of inference. After reviewing a result, the user may save a prediction record. The architecture routes that action through a prediction-log repository and prevents duplicate saves within the same session. This design avoids populating history with abandoned or repeated checks and gives the user an explicit decision over whether an image and its output are retained.

A history entry may contain the image thumbnail or reference, predicted label, confidence, timestamp, processing time, FPS, and the model identifier associated with the prediction. Retaining

The history screen provides filtering, export, and deletion controls. Export supports transfer of selected records for consultation, demonstration, or controlled analysis. Deletion supports user control over retained images and prediction metadata. The screen also presents aggregate runtime fields; these summaries are application diagnostics rather than research-performance estimates. When user feedback or a corrected label is available, it should be stored as a separate field from the original model output so that the prediction record is not overwritten retrospectively.

The architecture distinguishes a local repository from a Firebase-backed repository. Local SharedPreferences are identified for supported local records, while Firebase/Firestore is used when the cloud repository is configured. The available evidence does not define collection names, document schemas, storage paths, or security rules, so the implementation is described at the service boundary only. Network failure should not invalidate an on-device prediction; it should instead mark the record as unsynchronized or retain it locally where the supported flow allows.

Figure 5.4 connects history management with runtime configuration. The first screen shows a saved inspection, summary fields, and export/delete actions. The second shows account information, the configured model filename, CPU execution, image size, and thread count. Together, they expose the relationship between a stored prediction and the runtime configuration that produced it.

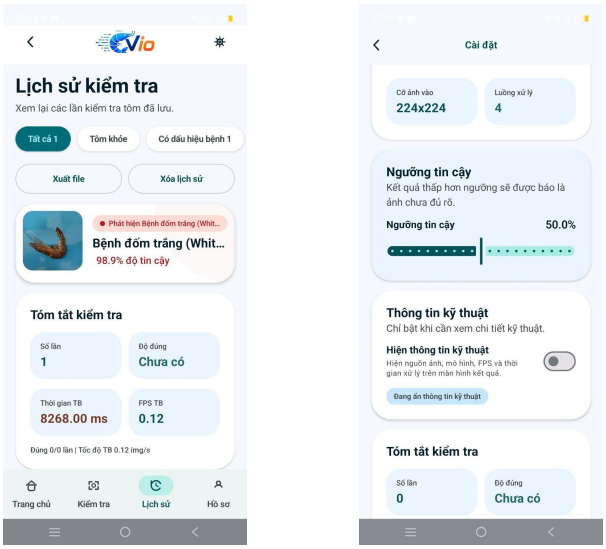


Figure 5.4. History and runtime-settings screens used in the mobile prototype.

The historical screen records approximately 8.27 s and 0.12 FPS for a single prototype run. This value is retained only as a UI-level observation and is not used as the final mobile benchmark. Chapter VI instead reports branch-aware operational runtime from three Android devices using the same application build and two-model release bundle. Because the application timer may include image preprocessing, interpreter activity, postprocessing, mask decoding, and UI-related work, these measurements are reported as application-recorded elapsed runtime rather than isolated neural-network latency.

2.5 End-to-End Data Flow Table

The complete executable path is summarized in Table 5.2. The flow distinguishes the classifier, which runs for every accepted image, from the conditional segmentation branch. The branch decision is an application-control rule derived from model output and confidence; it is not a statement of veterinary ground truth.

Table 5.2. End-to-end data flow

Stage	Input	Processing component	Output / next destination
Acquisition	Camera frame or gallery item	CameraX/gallery selection and explicit user confirmation	Decodable source bitmap; next: validation.
Validation	Source bitmap or URI	Decode, accessibility, dimensions, orientation, and format checks	Validated RGB image or bounded error.
Classification preparation	Validated image	Foreground preparation; resize to 224×224; normalization; NHWC tensor construction	$[1, 224, 224, 3]$ FP tensor; next: classifier.
Classification inference	Model-ready tensor	Final FP32 classifier executed on CPU/XNNPACK	Four scores; next: decision.
Decision	Decoded scores	Fixed class mapping, ranking, confidence extraction, and application threshold	Healthy/below-threshold: result builder. Disease above threshold: segmentation dispatcher.
Segmentation	Disease-triggered decision plus validated image	640×640 preprocessing; final YOLO11n-seg + CA→SimAM FP16 artifact; detection/prototype decoding; BG/WSSV filtering; NMS	Mask instances or bounded mask error; next: overlay compositor.
Presentation	Prediction object plus optional mask	Compose UI, bounded explanation, ranked alternatives, and runtime fields	User-readable screening result; next: review/save.
Persistence	User-approved result	Prediction-log repository, duplicate prevention, and local/cloud routing	Versioned history record or synchronization state.
Feedback / maintenance	History record plus correction when supplied	Preserve original model output; attach feedback; identify model/app version	Traceable maintenance case.

The staged design makes the conditional runtime path explicit: every usable record includes classification, whereas only disease-triggered records include segmentation. Device-level mean runtime therefore depends on branch composition and should not be interpreted as a strict hardware ranking when the uploaded images or the number of segmentation-triggered cases differ across devices.

Traceability metadata are attached when the result object is created and retained when it is saved. A later failure can therefore be associated with the source image, application build, classifier artifact, segmentation artifact when invoked, runtime configuration, and persistence state.

3. Deployment Strategy

3.1 Model Conversion and Packaging

Deployment freezes a two-artifact release bundle rather than a classifier plus a provisional segmentation interface. The approved classifier is exported as an FP32 TensorFlow Lite/LiteRT artifact, while the selected YOLO11n-seg + CA→SimAM checkpoint is packaged as the 5.65 MB FP16 segmentation artifact used by the application. Both artifacts are tied to the application build, decoder version, class mappings, and runtime configuration.

Conversion verification is divided into structural and behavioral checks. Structural checks inspect input/output shape, data type, supported operators, precision, and file identity. Behavioral checks use known images to verify preprocessing parity, class-index mapping, confidence decoding, segmentation output compatibility, and overlay alignment. Loading a file successfully is not sufficient evidence of numerical or semantic parity.

For the final classifier, the verified mobile contract is [1,224,224,3] input and [1,4] output in fixed class order. For the final segmentation model, the application uses a separate 640×640 RGB preprocessing path and the matching detection/prototype decoder. The segmentation model is not fed the 224×224 classifier tensor.

Operator compatibility remains conservative. The common demonstrated path is on-device CPU execution with four threads and XNNPACK/LiteRT acceleration; GPU, NNAPI, and iOS performance are not inferred.

3.2 Android Runtime Deployment

The application is a native Android project implemented primarily in Kotlin with Jetpack Compose, CameraX, Hilt, and view-model orchestration. Background execution prevents bitmap preparation and model invocation from blocking the presentation layer.

The three runtime devices reported in Chapter VI used the same application build, the same FP32 classifier artifact, the same FP16 segmentation artifact, and the configured four-thread CPU path. The evaluated phones were vivo Y100 (Android 15), Samsung A32 (Android 13), and POCO M7 Pro 5G (Android 16). The device study was Android-only; no iOS/iPhone build or runtime experiment was performed.

Each device recorded 21 usable runs using manually uploaded real-world shrimp photographs. The members were not required to replay an identical image-by-image order because the study prioritized operational runtime feasibility rather than a controlled hardware benchmark. The order and branch mixture therefore remain part of the interpretation boundary.

The application separates initialization, image-specific inference, segmentation-decoder, and persistence errors. A segmentation failure preserves the classification output; a cloud/persistence failure occurs after local inference and is not reported as a model failure.

Figure 5.5 shows the technical controls used for deployment verification. The confidence threshold controls application warning/dispatch behavior, while the model-information view exposes the configured filename, input/output tensors, class count, and serialized class order. The displayed 50% threshold is an

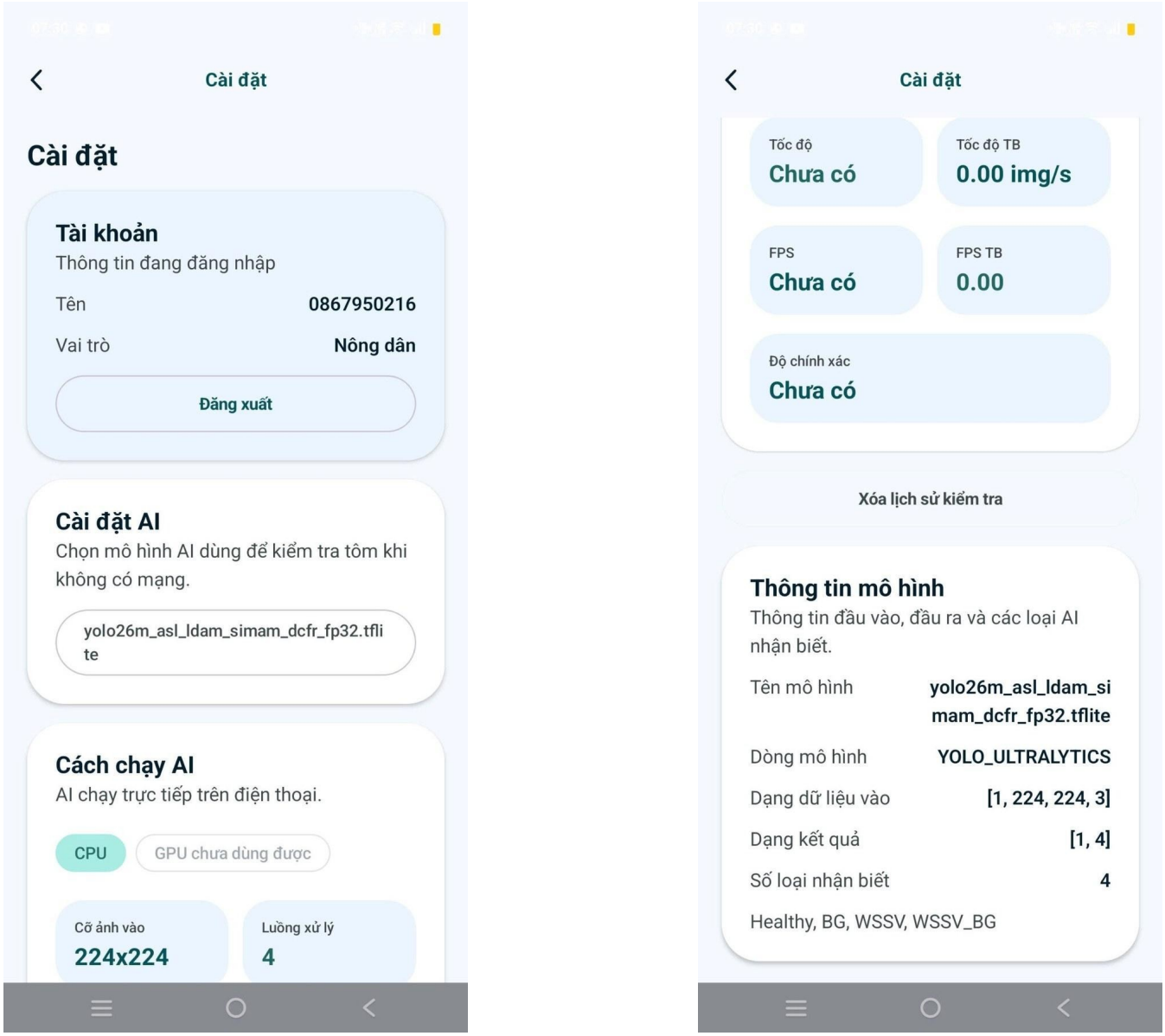


Figure 5.5. Model information and processing-summary screens used for deployment verification and debugging.

application setting rather than a research-metric threshold. Changing it can alter low-confidence messages and segmentation dispatch without retraining the model or changing offline benchmark definitions.

3.3 Firebase and Administrator-Side Services

Firebase/Firestore remains an application-data boundary for authentication, profiles, prediction history, administrator functions, and synchronization. Local inference is independent of Firebase availability. No cloud inference service is claimed.

The administrator branch reuses the same inference pipeline so that class mapping, preprocessing, and model bundle semantics are not duplicated. Administrator data access remains subject to the application

3.4 Deployment Procedure

The release workflow was updated to treat the classifier, the conditional segmentation artifact, the decoder, and the Android application as one versioned deployment unit:

1. Freeze the selected classifier and segmentation research checkpoints and record dataset/split provenance, class mappings, preprocessing, and input sizes.
2. Export `yolo26m_asl_ldam_simam_dcfr_fp32.tflite` and `yolo11n_seg_float16.tflite` using the approved precisions.
3. Inspect tensor shapes, dtypes, supported operators, file identity, and decoder compatibility for both artifacts.
4. Run known-image sanity inputs and verify classifier class-index mapping and segmentation output interpretation against the source checkpoints.
5. Package both artifacts with the Android build and bind the matching class maps, confidence handling, decoder version, and four-thread CPU configuration.
6. Verify the Healthy/below-threshold short-circuit path produces no disease mask.
7. Verify an above-threshold BG/WSSV/WSSV_BG output dispatches segmentation and produces BG/WSSV overlay semantics correctly.
8. Verify image orientation, preprocessing, ranked scores, mask alignment, and bounded error behavior on the target device.
9. Save results and verify history, duplicate prevention, export/deletion, and synchronization in the supported configuration.
10. Record application version, both model identifiers, release date, device/runtime configuration, and artifact hashes when available.
11. Retain the previous working application/model bundle for rollback.

This sequence distinguishes model conversion from deployed-system verification. A release is accepted only when both model artifacts and the conditional branch logic are valid in the same application build.

3.5 Deployment Interface Table

Table 5.3 consolidates the final mobile specification without duplicating numerical performance tables from Chapter VI.

Table 5.3. Final mobile model bundle and verification purpose

Item	Verified deployment specification	Verification purpose
Stage 1 model	YOLO26m-cls + ASL-LDAM + SimAM-DCFR	Connect mobile artifact to the selected classification configuration
Classifier artifact	`yolo26m_asl_ldam_simam_dcfr_fp32.tflite`; FP32	Identify exact classifier release; do not infer final parameter count from the plain YOLO26m reference
Classifier tensors	RGB NHWC [1,224,224,3] $\rightarrow$ [1,4]	Verify resize/layout and four-score decoder
Classifier classes	Healthy, BG, WSSV, WSSV_BG	Prevent class-index inversion across UI/history/export
Dispatch rule	Healthy or below threshold: stop; BG/WSSV/WSSV_BG above threshold: invoke segmentation	Ensure runtime branch semantics match the implemented pipeline
Stage 2 model	YOLO11n-seg + CA$\rightarrow$SimAM at P3/P4/P5	Bind selected attention model to the mobile decoder
Segmentation artifact	`yolo11n_seg_float16.tflite`; FP16; 5.65 MB	Verify final compact segmentation file used by the app
Segmentation input/classes	640$\times$640 RGB; BG and WSSV mask classes	Prevent accidental use of the classifier tensor or WSSV_BG as a mask class
Execution mode	Android on-device CPU; four threads; XNNPACK/LiteRT; GPU path disabled in retained configuration	Reproduce the common application runtime configuration
Platform boundary	Three Android devices evaluated; no iOS/iPhone test	Prevent unsupported platform claims

Item	Verified deployment specification	Verification purpose
Logged output	Predicted class, confidence, branch-dependent elapsed time, model identifier/history fields	Support operational traceability; prediction is not a veterinary ground-truth label
Timing boundary	Application-recorded elapsed time; may include preprocessing/interpreter/postprocessing/UI overhead	Avoid mislabeling runtime as isolated pure network latency
Persistence	User-approved save; model/build metadata retained; local/Firebase routing where configured	Support maintenance, export, deletion, and regression analysis

The final deployment description now separates three different provenance categories that must not be conflated: (1) research training/reproducibility hardware, reported in Chapter IV as the accepted dual NVIDIA Tesla T4 environment; (2) notebook/GPU timing used for selected model-efficiency diagnostics; and (3) Android application runtime from real phones. Numbers may be compared only within their stated measurement context.

4. Maintenance and Scalability

4.1 Component Modularity and Replacement

The implementation remains modular: acquisition, preprocessing, classifier, conditional segmentation, presentation, persistence, and administrator services communicate through explicit interfaces. Replacement is valid only when tensor shape, class order, normalization, precision, decoder semantics, and release metadata remain compatible.

Because the production path is a two-artifact bundle, future replacement should be evaluated at both the component and cascade levels. A classifier update can change how frequently segmentation is dispatched even when the segmentation model is unchanged; a segmentation update can change runtime and mask behavior without changing classification. Regression review must therefore preserve branch-level runtime and semantic checks in addition to ordinary model metrics.

4.2 Model Versioning and Traceability

Every deployed model release should be represented by a versioned manifest. The manifest binds the Android artifact to the research evidence and prevents a model file from becoming an untraceable binary.

Table 5.4 defines the minimum release fields.

Table 5.4. Model-release metadata required for traceable deployment

Metadata field	Recorded content	Maintenance purpose
Model identifier	Architecture and release-specific name	Distinguish deployed artifacts and rollback targets
Training evidence	Dataset version, split policy, seed protocol, checkpoint source	Link the binary to the experiment that produced it
Semantic contract	Class order, mask classes, input size, preprocessing version	Prevent silent incompatibility with the Android decoder
Conversion record	Format, precision, converter version, tensor shapes, operator audit	Diagnose conversion and runtime incompatibilities
Integrity record	File size and checkpoint/artifact hash when available	Detect accidental replacement or corruption
Release record	Release date, Android application version, target device class	Reconstruct the deployed configuration
Evaluation summary	Primary classification and segmentation metrics referenced from controlled reports	Gate replacement without duplicating Part VI results in the application chapter
Runtime record	Device, CPU, thread count, delegate, warm-up policy, preprocessing timing scope	Make latency observations reproducible and bounded

Prediction history should retain the model identifier used for each check. When a user later corrects a label or reports a false positive, the maintenance team can associate the case with the exact release. Without this field, retraining data may combine outputs from incompatible model generations and make regression analysis unreliable.

4.3 Dataset and Feature Growth

New farms, cameras, acquisition conditions, external datasets, and improved masks can extend the evidence base, but they cannot be appended without control. New data must preserve source identity, acquisition metadata when available, and partition integrity. Images from the same physical

Adding a disease category requires coordinated changes across the full system. The annotation protocol must define the new class, the classifier output head must be retrained, the serialized class mapping must be updated, the mobile output tensor and decoder must be verified, and the result interface must receive new terminology and bounded explanatory text. If the class requires spatial evidence, the segmentation mask policy and overlay colours must also be updated. Existing history records must retain their original class schema rather than being reinterpreted under the new release.

Mask improvements require versioned annotation data. A revised polygon boundary or relabeled overlap changes the target distribution and can invalidate direct comparisons with an earlier checkpoint. The dataset release should therefore record the annotation version, label semantics, annotator or review process, and split manifest. Single-annotator masks remain a limitation until independent review or agreement analysis is available.

Expanded corruption conditions and new devices should be treated as additional test domains, not as evidence of universal farm generalization. A model that tolerates one synthetic blur or lighting transform may still fail under reflections, occlusion, underwater colour shifts, or an unseen camera pipeline. New conditions should enter the regression suite only after their transform or acquisition protocol is documented.

4.4 Maintenance and Regression Verification

Maintenance is triggered by evidence, including repeated low-confidence outputs, recurrent false positives on Healthy images, user-corrected labels, poor mask alignment, difficult lighting, blur, background changes, conversion incompatibility, runtime slowdown, or Firebase failure. Cases should be stored with their original prediction, model version, source image subject to consent, and failure category. The original output must not be overwritten by the correction.

A replacement model should be compared with the deployed artifact under both research and application criteria. Classification review includes Macro-F1, classwise precision and recall, confusion behavior, and Healthy-versus-disease errors. Segmentation review includes the relevant mask metrics, disease missed-image behavior, false positives on verified negative images, mask count, and qualitative boundary inspection. Deployment review includes file size, conversion success, operator support, preprocessing parity, device latency, memory behavior, mask-decoder compatibility, and UI class mapping.

Table 5.5 defines a compact replacement gate. No single metric is sufficient: a higher offline score does not justify deployment if conversion fails, latency becomes unusable, or the Android class map is incompatible.

Table 5.5. Maintenance and regression verification matrix

Verification layer	Required evidence	Failure signal	Release decision
Classification	Controlled Macro-F1, classwise behavior,	Minority-class regression or increased Healthy/	Reject or retain for further analysis

Verification layer	Required evidence	Failure signal	Release decision
Segmentation	Mask metrics, missed-image behavior, negative-image false positives, mask review	Missing disease regions, spurious masks, or decoder mismatch	Reject spatial-evidence release
Conversion	Tensor metadata, operator audit, known-image parity	Unexpected shape/type, unsupported operator, or changed class order	Block Android packaging
Device runtime	Initialization, preprocessing, inference, memory, and thermal observations	Crash, severe latency regression, unstable output, or memory pressure	Optimize, change artifact, or roll back
Interface	Labels, ranked scores, threshold behavior, overlay alignment, history fields	Incorrect terminology, stale metadata, or misaligned mask	Block application release
Persistence	Save, duplicate prevention, export, deletion, offline state, synchronization	Lost record, unintended duplication, failed deletion, or hidden sync error	Fix repository flow before release

Regression verification should use the previous deployed version as the reference and retain the same known-image sanity set. Device runtime should be measured after warm-up and with the timing scope stated explicitly. A change is accepted only when its scientific and application effects are both understood.

4.5 Security, Privacy, and Operational Boundaries

The application handles account information, shrimp images, model outputs, and history records. Users should retain control over whether a result is saved, exported, synchronized, or deleted. The architecture’s explicit save action supports this principle. Deletion must remove the record from the storage destinations supported by the current configuration rather than only hiding it from the screen.

Local inference and cloud synchronization are separate trust boundaries. The classifier can execute without transmitting the image to a remote inference service. Firebase-dependent account and synchronization functions require network and authentication state. When those services are unavailable, the application should report the persistence state clearly and avoid repeated silent retries that create duplicate records.

Administrator functions require stricter access than farmer-facing screening. The system description confirms a basic dashboard and data-review branch but does not provide evidence for detailed authorization rules. Therefore, the design requirement is role-restricted access to profiles, prediction logs, consented images, and model settings; no claim is made about a specific

4.6 Current Limitations and Future Hardening

The revised application evidence is stronger than the earlier single-device prototype description: the final conditional classifier–segmentation cascade was exercised on three Android phones using one common application build, one FP32 classifier artifact, one FP16 segmentation artifact, and the configured four-thread CPU path. Nevertheless, the measurements remain an observational runtime-feasibility study rather than a controlled image-by-image hardware benchmark because uploaded images and branch compositions were not standardized across devices.

No iOS/iPhone implementation or runtime measurement was performed. GPU and NNAPI delegate behavior on Android is also not established. Device-level timing is application-recorded elapsed time and may include preprocessing, interpreter initialization, postprocessing, mask reconstruction, and UI overhead; it therefore must not be reported as pure model latency.

The 21 usable field photographs were collected from the three team-owned Android cameras and cover varied illumination, orientation, background, viewpoint, and image softness. They support operational execution under real capture variation but are not a field-accuracy benchmark. The shrimp did not receive veterinary/laboratory diagnosis, and the runtime logs contain no verified ground-truth correctness labels. Predicted classes were used only to determine which application branch executed; no mobile accuracy, sensitivity, specificity, or disease prevalence is claimed.

The classification release should continue to be audited with deterministic preprocessing tests, known-image parity checks, and artifact hashes. Exact final classifier parameter count and serialized-size metadata should be reported only from the released SimAM-DCFR artifact rather than inherited from the unmodified YOLO26m reference.

The segmentation model is now operationally verified in the Android cascade, but its scientific limitations remain: the disease masks are project-authored rather than independently pathologist-validated, and formal inter-annotator agreement is unavailable. Future hardening should prioritize decoder unit tests, signed/hashed model manifests, controlled same-image cross-device timing, delegate-specific profiling, repository synchronization tests, explicit authorization review, and regression suites covering known Healthy false-positive and difficult-capture cases.

The system remains a preliminary image-based screening tool. It does not prescribe treatment, replace laboratory confirmation, or transform model confidence into a clinical probability.